

6.1020 CHEAT SHEET

① Static Checking

Static types → Compile time

Dynamic types → Runtime

Static checking

Bug automatically found before program is ran

Dynamic checking

Bug found when code is executed

No checking

Doesn't help find errors at all

Immutable → can't be changed once they have been created (ex. strings, TS → Readonly Array)

Mutable → changing contents (ex. arrays TS: push(), splice())

Reassignment → changing where a variable points To make reference reassignable → use keyword const

② Testing

Systematic testing: correct (test cases are legal clients of spec), thorough, small

Unit and Integration Testing, Iterative Test-first Programming

Combination of modules

Repeating

Debugging:

• setting breakpoints • continue
• step over (skip) • step into (go into)

Test-First Programming →

1. Specification
2. Tests (to exercise spec)
3. Implementation

Partitions → subdomains should be:

- disjoint
- complete
- nonempty

Include boundaries

(common off-by-one mistake)

- has to have legal inputs for test suite

Automated Unit Testing → ex. Mocha it() → name of test, body of test

Black Box Testing: choosing test cases only from spec, not implementation

Glass Box Testing: choosing test cases with knowledge of how function is implemented.

Coverage: statement coverage < branch coverage < path coverage

Automated Regression Testing: take input of elicited bug to use as test case.

③ Code Review

Purposes: Improving code, Improving Programmers

• DRY (Don't Repeat Yourself) duplicate code is bad

• Comment where needed

• Fail fast - Reveal bugs as early as possible

• Avoid magic numbers

• Use good names, camelcase

• Functions should return, not print results

• Avoid special-case code

Kinds of variables:

• local variable (in function)

• instance variable (in instance of object)

• global variable (outside any function or object)

Want to use global constants typically so that we don't have variables accessible & changeable from everywhere

④ Specifications

Behavioral equivalence - ability to substitute one implementation for another w/o affecting correctness

CLIENT

satisfies precondition input

satisfies postcondition output

* output unconstrained if precondition is violated!

IMPLEMENTER

Specification firewall

Computation

@param @returns

@throws (exceptions)

Problem: Risk of Mutation

Defensive Copying, Immutability (more efficient solution)

⑤ Designing Specs

• **Deterministic** (one input → one output), **underdetermined** (some inputs allow multiple valid outputs)

• **Operational** - give a series of steps that the method performs (ex. pseudo code descriptions)

• **Declarative** (preferable) - no steps, just properties of outcome & how its related to initial state.

• **Stronger**: P stronger than Q if set of states that match P is a strict subset of those that match Q.
(weaker) more constrained weaker precondition, stronger post → stronger spec

⑥ Abstract Data Types

Requires: Precondition

Effects: Postcondition

Abstraction, Modularity, Encapsulation, Information Hiding, Separation of Concerns

Representation Independence: Use of abstract type independent of its representation.

Benevolent Side-Effect: Implementation is free to mutate a rep value as long as it continues to map to same abstract value.

- Creators - create new objects of the type (ex. factory-creator, new Object())
- Producers - one or more ^(input) existing objects → outputs new objects of the type (ex. concat (string × string → string))
- Observers - accessors/getters
- Mutators - change objects, setters

* ADT invariants replace preconditions

```
//Rep Invariant
//Abstraction Function
// Safety from Rep Exposure
```

Rep value : r

$R1(r)$ is true IFF r is mapped by A_F

Invariant:

⑧ Defining ADTs with Interfaces, Generics, Enums, & Functions

- safe for primitives (ex. number, string, boolean)
- not safe for user-defined object types (no knowledge of AF)

Hash Functions: set element type or map key type have to be hashable. (any type using `==` for behavioral equality is safe)

Things to be careful of:

- use **this** when referencing instance variables within an ADT.
- make sure for subtypes $\rightarrow$ all methods are written
- preconditions: check if anything should be nonnegative.
postconditions: reference preconditions (i.e. radius 'radius')

Grammar:
Find patterns
of wrapping, lowest
precedence at
outermost.

Kinds of Operations:

- **Creators** - create new objects of the type (ex. factory-creator, new Object())
 - **Producers** - one or more ^(input) existing objects $\rightarrow$ outputs new objects of the type (ex. concat (string x string $\rightarrow$ string))
 - **Observers** - accessors/getters
 - **Mutators** - change objects, setters
- *ADT invariants replace preconditions

Recursive Data Type:

Duck ex. $\text{ImDuck} = \underbrace{\text{loudDuck}(\text{next: ImDuck})}_{\text{variants of ImDuck}} + \underbrace{\text{QuietDuck}}_{\text{variants of ImDuck}}$

Concurrency:

For asynchronous functions $\rightarrow$ make sure keyword **async** in function signature, and return type is a **Promise** $\langle \rightarrow \rangle$.

When calling async functions, make sure to **await** to get result.

Reference Equality: 2 references point to same storage in memory

Value Equality: tests if 2 objects represent same value

Immutable Types $\xrightarrow{\text{same for immutable}}$ **Mutable Types**
Immutable built-in $\rightarrow$ $===$
Immutable Object types: `equalValue()` ($===$ too strong)
NOT safe for Set elems + Map keys

- **Abstraction Function**
 $\text{AF}(a) = \text{AF}(b)$
- **Observation** $\rightarrow$ can't be distinguished by calling any operations of the ADT.

- **Observational Equality** $\rightarrow$ test if they "look" the same
equalValue() pure observation, no mutation
- **Behavioral Equality** $\rightarrow$ 2 references "behave" the same (in this & all future states)
 $===$

Equivalence Relation:

Reflexive $(t, t) \in E \ \forall t \in T$

Symmetric $(t, u) \in E \Rightarrow (u, t) \in E$

Transitive $(t, u) \in E \wedge (u, v) \in E \Rightarrow (t, v) \in E$

Deep Equality:

- safe for primitives (ex. number, string, boolean)
- not safe for user-defined object types (no knowledge of AF)

Hash Functions: set element type or map key type have to be **hashable**. (any type using $===$ for behavioral equality is safe)

Problem: Risk of mutation

Defensive Copying, Immutability (more efficient solution)

⑤ Designing Specs

- **Deterministic** (one input $\rightarrow$ one output), **underdetermined** (some inputs allow multiple ^{valid} outputs)
- **Operational** - give a series of steps that the method performs (ex. pseudo code descriptions)
- **Declarative** (preferable) - no steps, just properties of outcome + how its related to initial state.
- **Stronger**: P stronger than Q if set of states that match P is a strict subset of those ^{match} Q.
(weaker) $\uparrow$ more constrained weaker precondition, stronger post $\rightarrow$ stronger spec